PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

(PUC-CAMPINAS)

Curso: Engenharia de Computação		**Disciplina**: Estrutura de Dados		
Período: 04	**Turma**: 01	**Sala**: S01 | H15	**Turno**: Integral	**Data Entrega**: 10/10/2025
Integrante(s): **Artur Yano Contarelli** 24014303 **Lucas Kitsuta Sabino** 24001484 **Rafael Martiniano Nogueira Filho** 24008538 **Tiago Alves Rodrigues** 24001623				
Orientador(a): **Lúcia Filomena de Almeida Guimarães**				

Estrutura de Dados | Projeto 1

Gerenciamento de Atendimento em Clínica Veterinária

Manipulação de Fila

Sumário

1. **Introdução** 02
2. **Desenvolvimento** 02
    a. **Geração de Números Aleatórios** 03
    b. **Estrutura de Dados e Adaptação da Biblioteca de Fila** 04
    c. **Desafios e Soluções Implementadas** 04
3. **Bibliografia** 05

1. Introdução:

Nosso projeto tem como objetivo principal a aplicação prática dos conceitos de manipulação dinâmica de memória, ponteiros e estruturas de dados encadeadas, estudados na disciplina de Estrutura de Dados. O trabalho consiste no desenvolvimento de um sistema em Linguagem C para simular e gerenciar a fila de atendimento de uma clínica veterinária.
O sistema gerencia duas filas de atendimento distintas: uma para casos de emergência e outra para atendimentos normais. A lógica de negócio implementada garante que animais em estado de emergência tenham prioridade no atendimento, sendo atendidos antes dos demais que aguardam na fila normal. Para cada pet, o sistema armazena informações essenciais como ID, nome, espécie, idade e data de nascimento.

2. Desenvolvimento

O desenvolvimento foi estruturado com base nos requisitos funcionais e técnicos descritos no documento de orientação do projeto. A seguir, detalhamos as soluções implementadas e os desafios encontrados.
Passo a Passo:

- Distribuir funções e prazos para a criação do código
- Organizar isso de forma que todos possam entender e ficar atualizados com o código outro.
- Ao longo dos códigos, verificar se é otimizado e encaixa com os outros
- Verificar o funcionamento como um todo ajustando possíveis erros
- Descrever o relatório conforme isso é feito.

Organização de funções:

- Inserir um pet (colocar na fila correta)
- Atender um pet (tirar da fila)
- Buscar um pet pelo nome/ID
- Imprimir um relatório com as informações de todos os pets de cada lista
- Imprimir informações do próximo pet a ser atendido
- Imprimir todos os pets que já foram atendidos (guardar aqueles que saem da lista)
- Deixar a interface mais intuitiva e agradável.
- Terminar o relatório.

2.1 Geração de Números Aleatórios

Como números aleatórios funcionam:

Por natureza, computadores geram resultados previsíveis com base em entradas. Para gerar números aleatórios, eles usam algoritmos chamados Geradores de Números Pseudoaleatórios. Esse gerador requer uma "semente" (um valor inicial) para começar uma sequência de números. O número que o PRNG gera é o resultado de um procedimento algébrico que pode ser determinado → EX: seed = 0, a =5, c=8 → (seed X a + c) % 7 == 1, alterando a seed de acordo com um algoritmo, para que o número se torne pseudo aleatório. Desse modo, é possível estabelecer a seed como sendo o relógio do computador, que vai mudando a cada segundo, adquirindo a impressão de mudança e aleatoriedade.

Sendo assim, ao incluir as bibliotecas <stdlib.h> e <time.h>, para usarmos os métodos srand(), rand() e time(), é possível inicializar com semente baseada no relógio → srand(time(NULL)) e limitar seu intervalo → (rand() % (max - min + 1)) + min).

Para o nosso código:

Um dos requisitos do projeto era a geração de um ID único e aleatório para cada pet, em um intervalo entre 100 e 999. Dessa forma, usamos Semente de Randomização: Na função main, a função srand(time(NULL)) é chamada no início da execução. A utilização do relógio do sistema (time(NULL)) como semente garante que a sequência de números aleatórios seja diferente a cada vez que o programa é executado. Depois, foi necessário gerar e verificar o ID para não gerar repetidos: Foi criada a função gerarID, que entra em um laço de repetição. Dentro do laço:

- Um número é gerado no intervalo desejado usando a expressão rand() % 900 + 100.
- A função searchId é chamada para verificar se o ID gerado já existe em qualquer uma das três filas (filaEmergencia, filaNormal e filaAtendidos).
- O laço persiste até que um ID inédito seja gerado, garantindo a não repetição de identificadores, conforme exigido.

2.2. Estrutura de Dados e Adaptação da Biblioteca de Fila

Para a gestão dos atendimentos, o sistema utiliza obrigatoriamente o Tipo Abstrato de Dados (TAD) Fila. A biblioteca de Fila, fornecida como base, foi utilizada para instanciar três filas distintas no sistema:

1. filaEmergencia: Armazena os pets cadastrados com prioridade de emergência (prioridade 0).
2. filaNormal: Armazena os pets cadastrados com prioridade normal (prioridade 1).
3. filaAtendidos: Funciona como um histórico, armazenando os pets que já foram removidos das filas de espera e atendidos.

A principal adaptação não ocorreu na lógica da biblioteca em si (FILA_arguicao.h), uma vez que modificamos apenas o tipo de objeto utilizado (cada nó passou a conter a informação Pet, ao invés de um inteiro). Ademais, a biblioteca genérica foi utilizada para criar e gerenciar múltiplas instâncias de filas dentro do arquivo principal (clinica_veterinaria_arguicao.c). Nesse arquivo, utilizamos métodos para manipulação e navegação de fila, para realizar as ações necessárias, enquanto a lógica de priorização—verificar primeiro a fila de emergência antes da normal—foi implementada nas funções de atendimento do programa principal, como a atenderPet(). A estrutura Pet foi definida no arquivo de cabeçalho para conter todos os dados necessários, incluindo um campo prioridade que determina em qual fila o animal será inserido.

2.3. Desafios e Soluções Implementadas

Durante o desenvolvimento, um dos principais desafios foi a manipulação correta das múltiplas filas e a garantia da lógica de prioridade. A função atenderPet foi cuidadosamente desenhada para sempre verificar se a fila de emergência não está vazia (!VaziaFila(filaEmergencia)) antes de tentar atender um pet da fila normal. Isso assegura que a regra de prioridade seja sempre respeitada.

Outro ponto de atenção foi a função de busca (buscarPet). Ela foi projetada para procurar o pet em todas as três filas, informando ao usuário a localização e o status do animal (aguardando em emergência, aguardando em fila normal ou já atendido), cumprindo o requisito de busca completa. A busca por nome utiliza a função strstr, permitindo encontrar um pet mesmo que o usuário digite apenas uma parte do nome, e foi implementada de forma a listar todos os resultados encontrados, caso haja mais de um pet com o mesmo nome.

A interface com o usuário foi construída de forma modular, com um menu principal (mostrarMenu) que direciona o fluxo do programa para as diferentes funcionalidades, como inserir, atender, buscar e imprimir relatórios, tornando o sistema organizado e de fácil utilização.

3. Bibliografia

Materiais de aula da disciplina de Estrutura de Dados.

ZHANG, Tony. How computers generate RANDOMNESS from math. [Vídeo]. YouTube, [s.d.]. Disponível em: https://www.youtube.com/watch?v=nBq4sFg3at0. Acesso em: 20/09?2025.

WIKIPEDIA. Hardware random number generator. [s.d.]. Disponível em: en.wikipedia.org .. Acesso em: 28/09/2025.

Bibliotecas C:

- UNIVERSIDADE DE SÃO PAULO. Instituto de Matemática e Estatística. time.h. [s.d.]. Disponível em: www.ime.usp.br. Acesso em: 30/09/2025.

- UNIVERSIDADE DE SÃO PAULO. Instituto de Matemática e Estatística. stdlib.h. [s.d.]. Disponível em: www.ime.usp.br. Acesso em: 26/09/2025.

- UNIVERSIDADE DE SÃO PAULO. Instituto de Matemática e Estatística. stdio.h. [s.d.]. Disponível em: www.ime.usp.br. Acesso em: 01/10/2025.

- UNIVERSIDADE DE SÃO PAULO. Instituto de Matemática e Estatística. string.h. [s.d.]. Disponível em: www.ime.usp.br. Acesso em: 27/09/2025.